IST 387: Introduction to Applied Data Science
Lab 3: Part 2 Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

Contents of the Notes:

- Sampling a Population
- Understanding Descriptive Statistics
- Using Histograms to Understand a Distribution
- Normal Distribution
- Case Study: Exploring LTR Distributions
- Important R functions

Preeti Jagadev

Sampling a Population

There is a fundamental aspect of working with all kinds of data: **whatever data you have, there's always more out there.** There's data that you might have collected by changing the way things are done or the way things are measured. There's future data that hasn't been collected yet and might never be collected. There is data that is inaccessible to us because of the expense or impracticality of collecting it. There's even data that we might have gotten using the exact same strategies we did use, but that would have come out subtly different just due to randomness. Whatever data you have, it is just a sample of what might be out there. This means that we can never ever 100% trust the data we have. We must remember that there is always uncertainty in data. A lot of power and goodness in statistics come from the techniques developed to help us understand that uncertainty and to know when to guard against putting too much stock in what a sample of data has to say. Remember that while we can always ***describe*** the sample of data we have, the real trick is to ***infer*** what the data could mean when generalized to the larger population of data that we don't' have. This is the key distinction between ***descriptive*** and ***inferential*** statistics.

Understanding Descriptive Statistics

- **Mean (Arithmetic mean):** Calculated by adding together all the observations and dividing this sum by the number of observations.

- **Median:** Median refers to the value of the data that splits the whole group in half, with half of the datapoints having higher values, and half having lower values.

Example 1:

- Consider the data set {3, 13, 2, 34, 11, 26, 47}.
- Arrange these data points in ascending order, i.e. {2, 3, 11, 13, 26, 34, 47}.
- The number of data points is 7, which is an **odd number.** Thus, the median is the number in the middle of {2, 3, 11, 13, 26, 34, 47}. which in this instance is 13 since there are three numbers on either side.

Example 2:

- Consider the data set {3,7,9,5}.
- Arrange these data points in ascending order, i.e. {3,5,7,9}.

Preeti Jagadev

- The number of datapoints is 4, which is an **even number**. If there is an even number of observations, then there is no single middle value. Thus, the median is defined to be the mean of the two middle values: so, the median of 3, 5, 7, 9 is (5+7)/2 = 6.

- **Range:** The range is a measure of dispersion- how spread out a bunch of numbers in a sample are- calculated by subtracting the lowest value from the highest value.

- **Mode:** It is the value that occurs most frequently in a sample of data. For example, if a set of numbers contains the following digits, 1, 1, 3, 5, 6, 6, 7, 7, 7, 8, the mode would be 7, as it appears the most out of all the numbers in the set.

- **Variance:** Variance is a measure of how data points differ from the mean. The formula to calculate variance is as follows:

$$S^2 = \frac{\sum (x_i - \bar{x})^2}{n - 1}$$

S^2 = sample variance

x_i = the value of the one observation

$\bar{x}$ = the mean value of all observations

n = the number of observations

Let's do some simple calculations with family ages, to understand the concept of variance.

```
myFamilyAges <- c(43, 42, 12, 8, 5)
mean(myFamilyAges)
```

The output is as follows:

```
> mean(myFamilyAges)
[1] 22
```

Table: Calculating the Variance

Who	Age	Age-Mean	$(\text{Age} - \text{Mean})^2$
Dad	43	43 – 22 = 21	21 * 21 = 441
Mom	42	42 – 22 = 20	20 * 20 = 400
Sis	12	12 – 22 = -10	-10 * -10 = 100
Bro	8	8 – 22 = -14	-14 * -14 = 196
Dog	5	5 – 22 = -17	-17 * -17 =289
		Total	1426
		Total/4	**356.5**

- **Standard Deviation:** Standard deviation is the square root of Variance.

The variance can be calculated using the following command:

```
var(myFamilyAges)
```

The output is as follows:

```
> var(myFamilyAges)
[1] 356.5
```

The standard deviation can be calculated using the following command:

```
sd(myFamilyAges)
```

The output is as follows:

```
> sd(myFamilyAges)
[1] 18.88121
```

Using Histograms to Understand a Distribution

Now, we will learn to use **hist( )** function to create histograms, which is one of the most common and useful statistical visualizations. This function takes in a vector of values for which the histogram is plotted.

Let us use the built-in dataset **airquality** which has "Daily air quality measurements in New York, May to September 1973."

```
str(airquality)
```

The output is as follows:

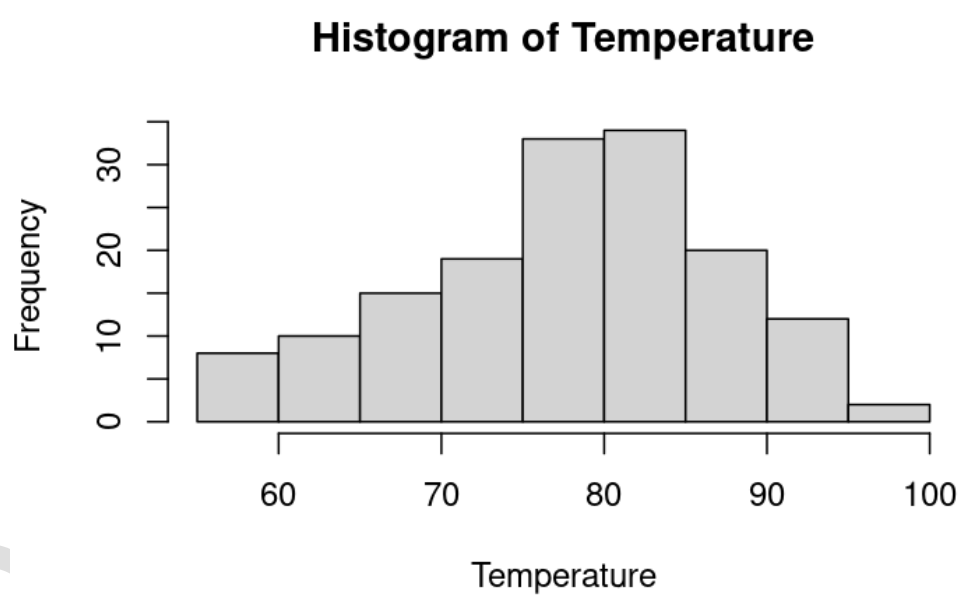
Preeti Jagadev

```
> str(airquality)
'data.frame':   153 obs. of  6 variables:
 $ Ozone  : int  41 36 12 18 NA 28 23 19 8 NA ...
 $ Solar.R: int  190 118 149 313 NA NA 299 99 19 194 ...
 $ Wind   : num  7.4 8 12.6 11.5 14.3 14.9 8.6 13.8 20.1 8.6 ...
 $ Temp   : int  67 72 74 62 56 66 65 59 61 69 ...
 $ Month  : int  5 5 5 5 5 5 5 5 5 5 ...
 $ Day    : int  1 2 3 4 5 6 7 8 9 10 ...
```

We will use the temperature parameter which has 154 observations in degrees Fahrenheit.

```
Temperature <- airquality$Temp
hist(Temperature)
```

The output is as follows:



A histogram is a specialized type of bar graph designed to show how often datapoints occur when they are organized into a set of equal sized ranges, called **bins.** The word frequency in the figure means how often a particular value or range of values occurs in a dataframe.

We can see above that there are 9 cells with equally spaced breaks. First, look along the Y-axis (the vertical axis on the left) for an indication of how often the data occur in each category. To know what each bar represents, look along the x axis (the horizontal axis at the bottom).

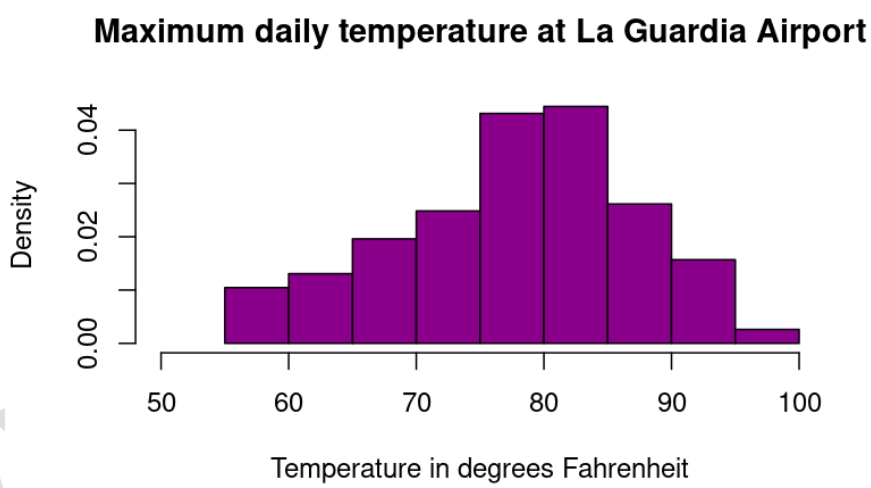
We can pass in additional parameters to control the way our plot looks. You can read about them in the help section **?hist**.

Some of the frequently used ones are, **main** to give the title, **xlab and ylab** to provide labels for the axes, **xlim and ylim** to provide range of the axes, **col** to define color etc.

Additionally, with the argument **freq=FALSE** we can get the probability distribution instead of the frequency.

```
# histogram with added parameters
hist(Temperature,
     main="Maximum daily temperature at La Guardia Airport",
     xlab="Temperature in degrees Fahrenheit",
     xlim=c(50,100),
     col="darkmagenta",
     freq=FALSE
)
```

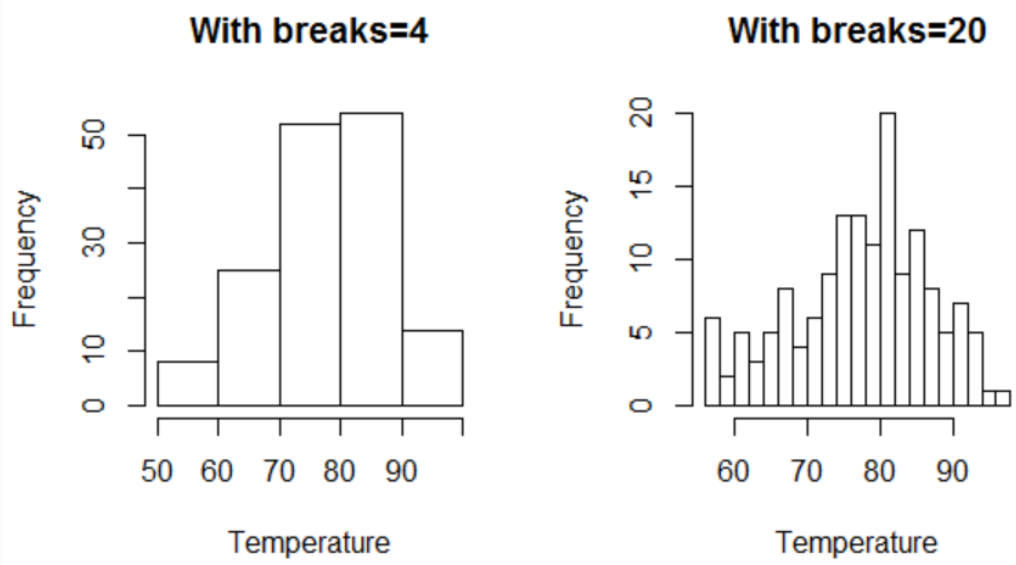
The output is as follows:



- **Defining the Number of Breaks**

With the **breaks** argument we can specify the number of cells we want in the histogram. However, this number is just a suggestion. R calculates the best number of cells, keeping this suggestion in mind. Following are two histograms on the same data with different numbers of cells.

```
hist(Temperature, breaks=4, main="With breaks=4")
hist(Temperature, breaks=20, main="With breaks=20")
```



In the above figure we see that the actual number of cells plotted is greater than we had specified.

Normal Distribution

Normal distribution, also known as Gaussian distribution, is a probability distribution that is often used in statistics and probability theory to describe continuous random variables.

The shape of a normal distribution is a bell curve, which is symmetrical and has a single peak at the center. The mean, median, and mode of a normal distribution are all equal, and the distribution is characterized by two parameters: the mean (μ) and the standard deviation (σ).

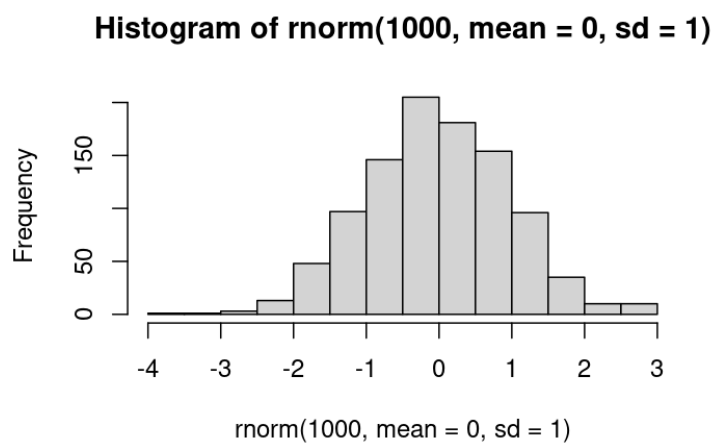
❖ The rnorm() function in R

In R, the normal distribution can be simulated using the built-in function **rnorm().** This function generates random numbers from a normal distribution with a specified mean and standard deviation. Here's an example of how to generate 1000 random numbers from a normal distribution with a mean of 0 and a standard deviation of 1:

```
# Generate 1000 random numbers from a normal distribution
data <- hist(rnorm(1000, mean = 0, sd = 1))
```

The output is as follows:

This code generates 1000 random numbers from a standard normal distribution (i.e., with a **mean** of 0 and a **standard deviation** of 1), stores the numbers in the variable data, and then plots a histogram of the data using the **hist()** function.



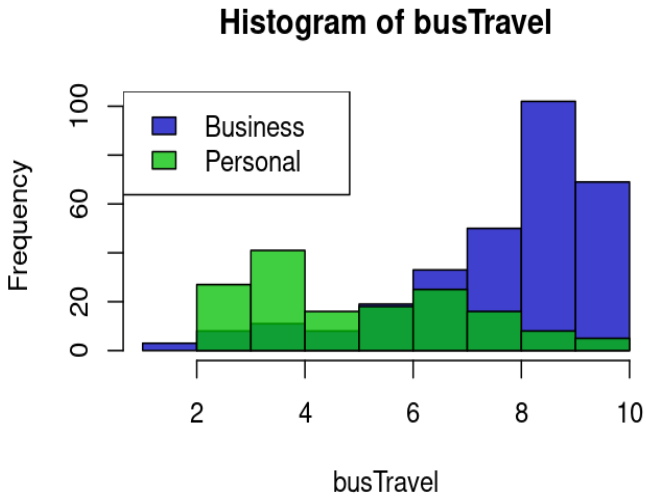
If you want to generate random numbers from a normal distribution with a different mean or standard deviation, simply adjust the values for the mean and sd parameters accordingly.

Case Study: Exploring LTR Distributions

Let's look at the distribution of LTR value across two key groups: business and personal travel.

```
install.packages("tidyverse")
library(tidyverse)
#load in the dataset
smallSurvey <- read.csv("smallSurvey.csv")

#Remove the NAs from the LTR column
smallSurvey <- filter(smallSurvey, !is.na(LTR))

#get the LTR for all business travels, using core R
bTrue <- smallSurvey$Type.of.Travel == "Business travel"
busTravel <- smallSurvey$LTR[bTrue]

#get the LTR for all personal travel, using tidyverse
perTravel <- smallSurvey %>% filter(Type.of.Travel == "Personal Travel") %>% pull(LTR)

#define two colors, including transperancy (alpha)
myBlue <- rgb(red=0, green=0, blue=0.75, alpha=.75)
myGreen <- rgb(red=0, green=0.75, blue=0, alpha=.75)

#generate two histograms
#adding the second histogram to the first
hist(busTravel, col=myBlue)
hist(perTravel, col=myGreen, add=TRUE)

#add a legend
legend("topleft", c("Business", "Personal"), fill=c(myBlue,myGreen))
```

The output is as follows:



We used three features to get this done:

1. When we defined the color (the fill color), we specified **alpha** (which controls transparency).

2. When using the **rgb** function, by default all the colors, including transparency, range from 0 to 1. A 0 value for alpha makes the color fully transparent, and a value of 1 makes the color opaque (not transparent).

3. For the second histogram, we used the **add=TRUE** parameter, which tells R to add the second histogram to the first histogram.

Important R Functions

- **filter( ):** Tidyverse function for subsetting
- **legend( ):** Adds a legend to a graphic
- **library( ):** Loads a package into memory
- **mean( ):** Calculates arithmetic mean
- **median( ):** Locates the median.

- **mode( ):** Tells the data type/mode of a object. Note: This is not the statistical mode.
- **mutate( ):** Tidyverse function to create, modify, or delete columns.
- **pull( ):** Tidyverse function to extract a column.
- **var( ):** Calculates the sample variance.
- **read_csv( ):** Import a comma separated data file.
- **rgb( ):** Define a color to use on a graph.
- **sd( ):** Calculates the sample standard deviation.
- **select( ):** Tidyverse function to subset columns.
- **slice( ):** Tidyverse function to subset rows.
- **str_replace_all( ):** Match and replace one or more instances of a string.
- **hist( ):** Produces a histogram.

Preeti Jagadev